

AI Assisted Coding

Roll No.2303A510A0(Vishnu Vardhan)

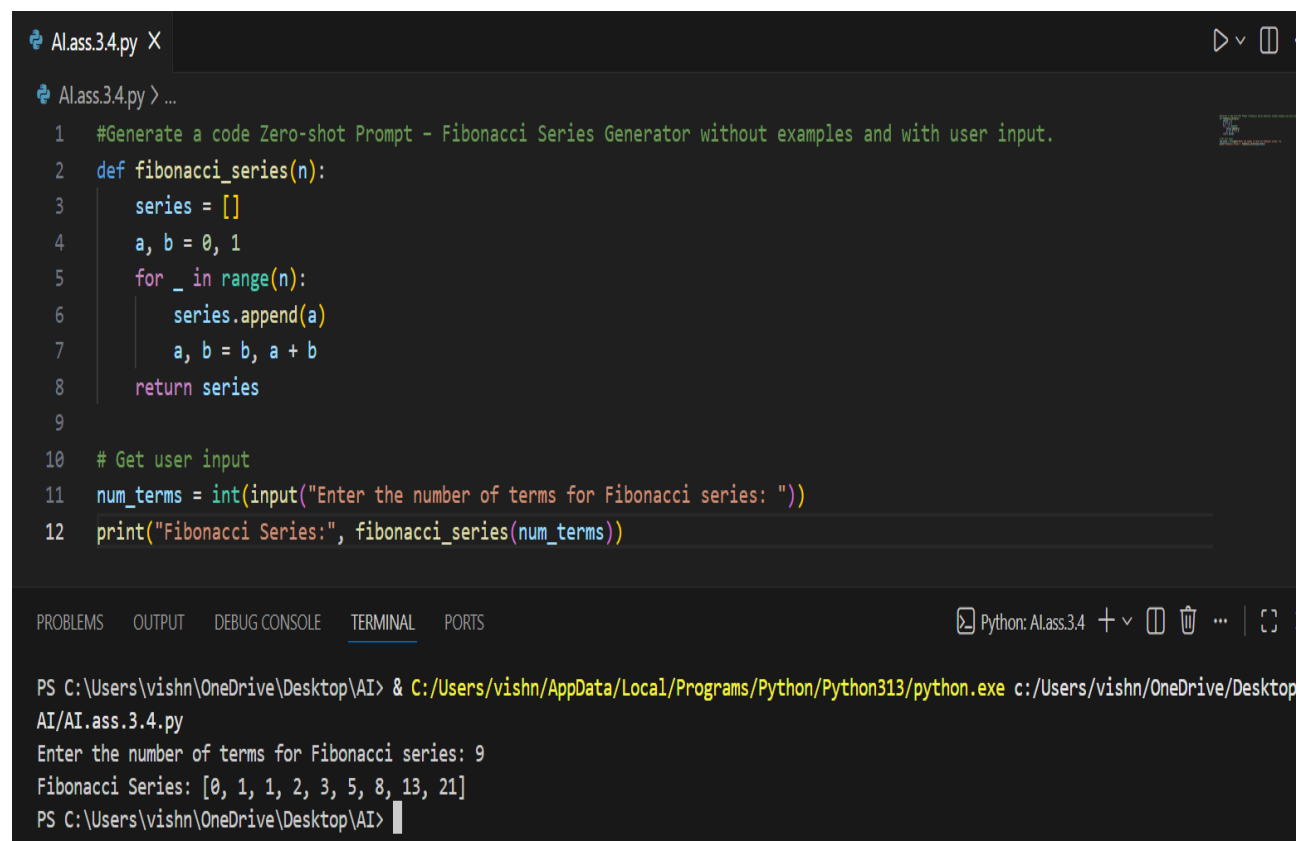
Batch-14

Assignment 3.4

Task 1: Zero-shot Prompt – Fibonacci Series Generator

Prompt: #Generate a code Zero-shot Prompt – Fibonacci Series Generator without examples and with user input.

Code & output:



```
AI.ass.3.4.py X
AI.ass.3.4.py > ...
1 #Generate a code Zero-shot Prompt - Fibonacci Series Generator without examples and with user input.
2 def fibonacci_series(n):
3     series = []
4     a, b = 0, 1
5     for _ in range(n):
6         series.append(a)
7         a, b = b, a + b
8     return series
9
10 # Get user input
11 num_terms = int(input("Enter the number of terms for Fibonacci series: "))
12 print("Fibonacci Series:", fibonacci_series(num_terms))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python: AI.ass.3.4 + v [ ] [ ] [ ] [ ] [ ] [ ]

PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
Enter the number of terms for Fibonacci series: 9
Fibonacci Series: [0, 1, 1, 2, 3, 5, 8, 13, 21]
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

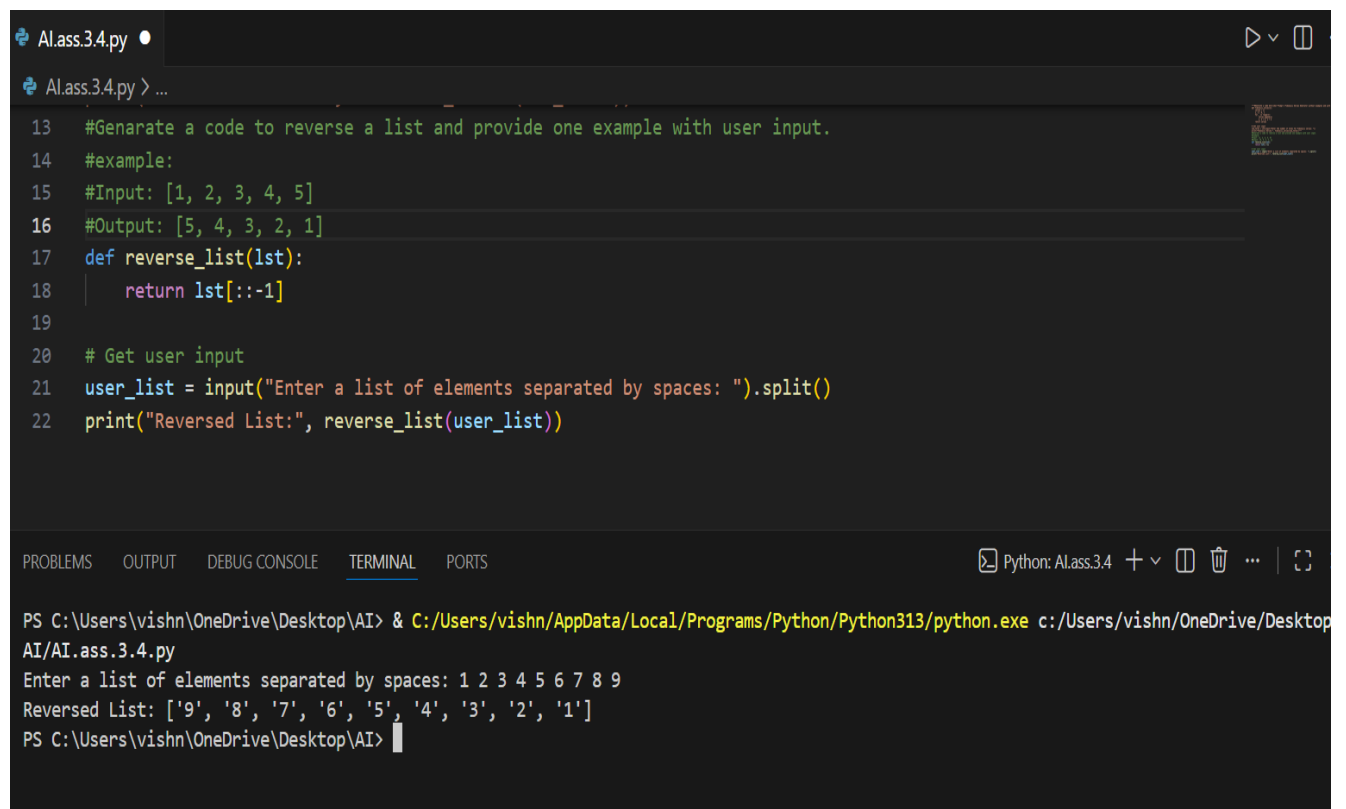
Observation: (Zero-shot Prompt):

GitHub Copilot correctly understood the requirement without any example and generated a valid Fibonacci function. It inferred the standard Fibonacci logic and sequence starting from 0 and 1. The output for $N = 7$ was correct as 0 1 1 2 3 5 8. This shows that zero-shot prompting works well for common and well-known problems, though output formatting may vary.

Task 2: One-shot Prompt – List Reversal Function

Prompt: #Generate a code to reverse a list and provide one example with user input

Code & output:



```
AI.ass.3.4.py •
AI.ass.3.4.py > ...
13 #Generate a code to reverse a list and provide one example with user input.
14 #example:
15 #Input: [1, 2, 3, 4, 5]
16 #Output: [5, 4, 3, 2, 1]
17 def reverse_list(lst):
18     return lst[::-1]
19
20 # Get user input
21 user_list = input("Enter a list of elements separated by spaces: ").split()
22 print("Reversed List:", reverse_list(user_list))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: AI.ass.3.4 + - [ ] [ ] [ ]
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
Enter a list of elements separated by spaces: 1 2 3 4 5 6 7 8 9
Reversed List: ['9', '8', '7', '6', '5', '4', '3', '2', '1']
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

Observation:(One-shot Prompt):

By providing a single input–output example, GitHub Copilot clearly understood the expected behavior of the function. The example guided Copilot to generate accurate and clean code for reversing a list, often using efficient methods like slicing. Compared to zero-shot prompting, the one-shot prompt reduced ambiguity and improved correctness. This shows that even one example can significantly enhance Copilot's accuracy and code quality

Task 3: Few-shot Prompt – String Pattern Matching

Prompt: #Generate a code for String Pattern Matching to check if a string starts with a capital letter and ends with a period. With user input.

#Example:

#Input: "Hello World."

#Output: True

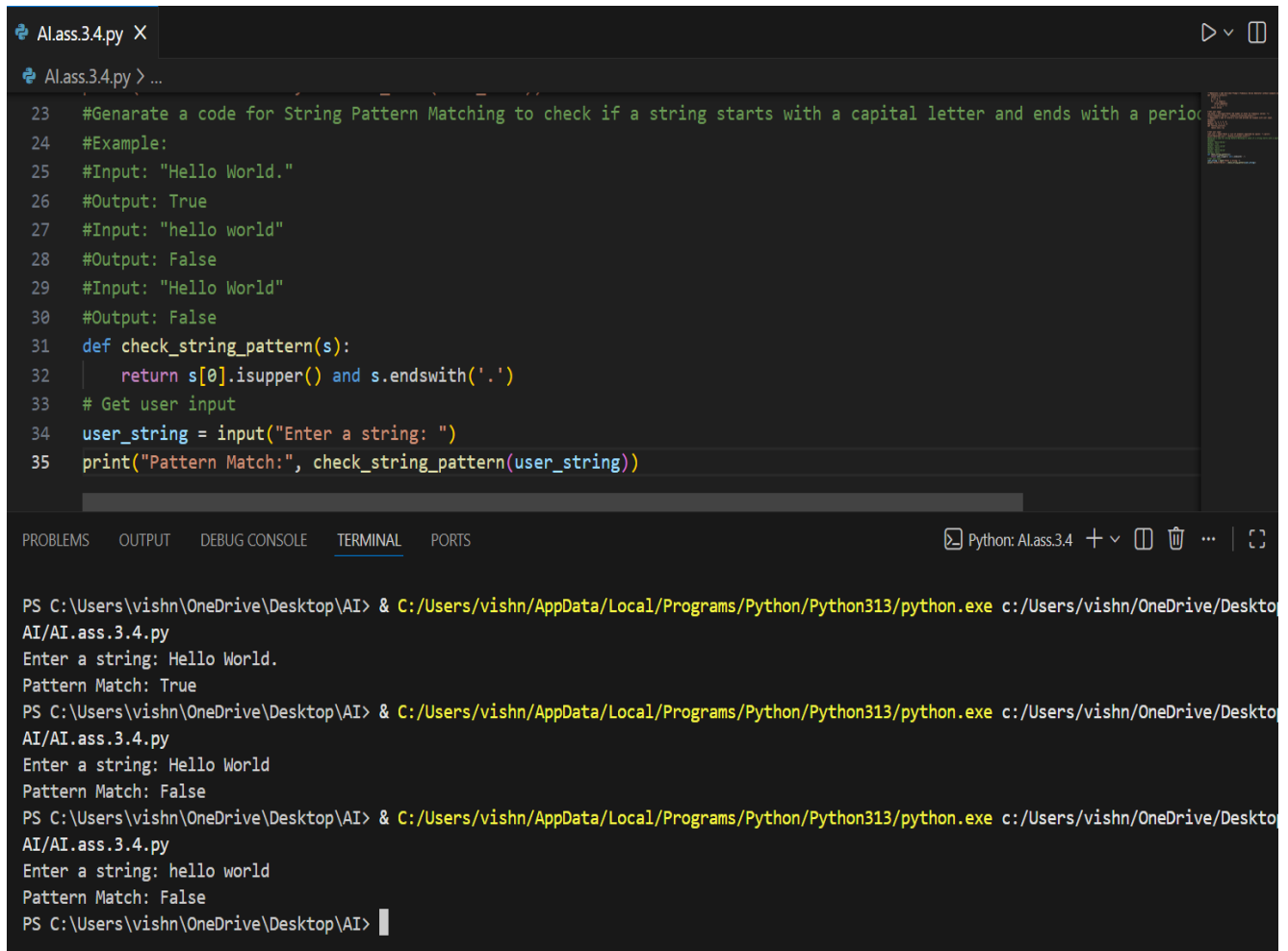
#Input: "hello world"

#Output: False

#Input: "Hello World"

#Output: False

Code & output:



The screenshot shows a VS Code editor window with a file named `AI.ass.3.4.py`. The code is a Python script that defines a function `check_string_pattern(s)` which returns `s[0].isupper() and s.endswith('.')`. It then prompts the user to enter a string and prints the result of the function. The terminal output shows three test cases: "Hello World." (True), "Hello World" (False), and "hello world" (False).

```
23 #Generate a code for String Pattern Matching to check if a string starts with a capital letter and ends with a period
24 #Example:
25 #Input: "Hello World."
26 #Output: True
27 #Input: "hello world"
28 #Output: False
29 #Input: "Hello World"
30 #Output: False
31 def check_string_pattern(s):
32     return s[0].isupper() and s.endswith('.')
33 # Get user input
34 user_string = input("Enter a string: ")
35 print("Pattern Match:", check_string_pattern(user_string))
```

Terminal Output:

```
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
Enter a string: Hello World.
Pattern Match: True
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
Enter a string: Hello World
Pattern Match: False
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
Enter a string: hello world
Pattern Match: False
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

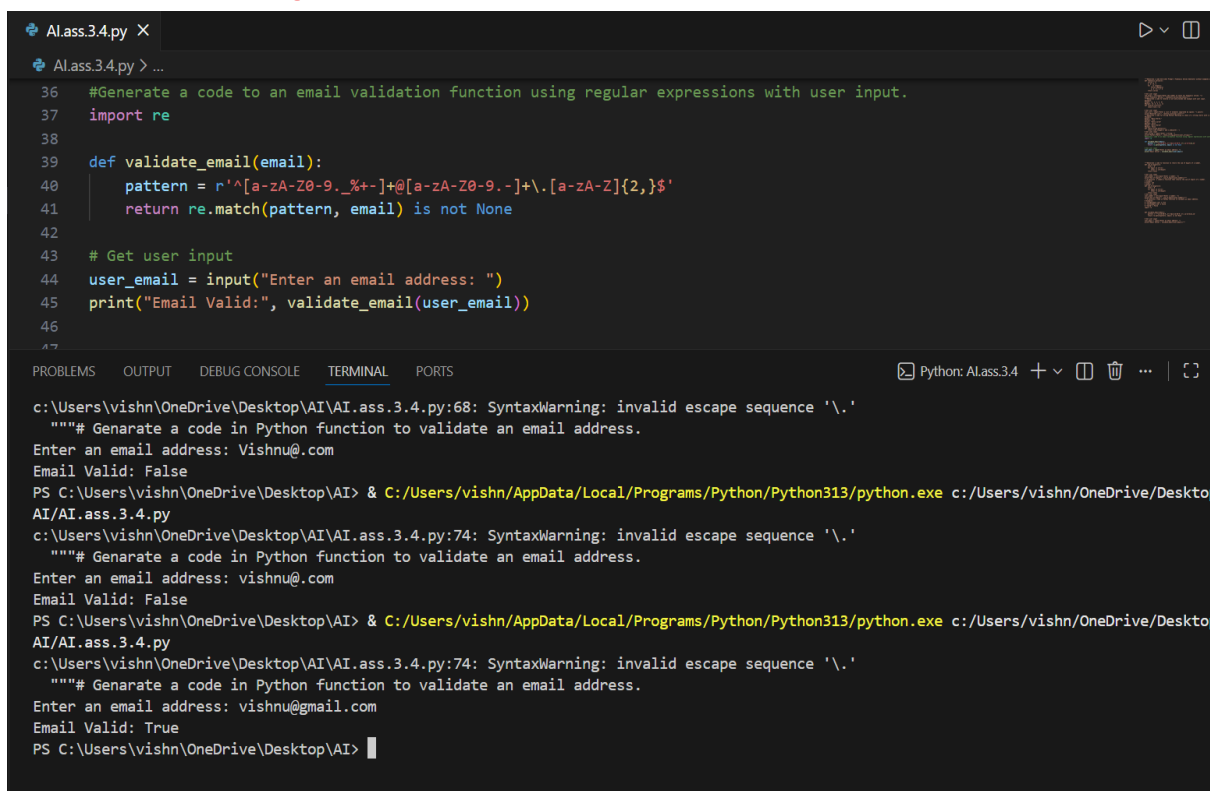
Observation: (Few-shot Prompt):

Providing multiple examples helped GitHub Copilot clearly understand the exact string pattern to be checked. The examples guided Copilot to correctly identify both conditions—starting with a capital letter and ending with a period. As a result, the generated `is_valid()` function was accurate and reliable. This demonstrates that few-shot prompting greatly improves precision by reducing ambiguity

Task 4: Zero-shot vs Few-shot – Email Validator

Prompt: #Generate a code to an email validation function using regular expressions with user input.

Code & Output:



```
AI.ass.3.4.py X
AI.ass.3.4.py > ...
36 #Generate a code to an email validation function using regular expressions with user input.
37 import re
38
39 def validate_email(email):
40     pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
41     return re.match(pattern, email) is not None
42
43 # Get user input
44 user_email = input("Enter an email address: ")
45 print("Email Valid:", validate_email(user_email))
46
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python: AI.ass.3.4 + v [ ] [ ] [ ] [ ]
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:68: SyntaxWarning: invalid escape sequence '\.'
    """# Genearate a code in Python function to validate an email address.
Enter an email address: Vishnu@.com
Email Valid: False
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop\AI\AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:74: SyntaxWarning: invalid escape sequence '\.'
    """# Genearate a code in Python function to validate an email address.
Enter an email address: vishnu@.com
Email Valid: False
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop\AI\AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:74: SyntaxWarning: invalid escape sequence '\.'
    """# Genearate a code in Python function to validate an email address.
Enter an email address: vishnu@gmail.com
Email Valid: True
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

Prompt: # Genearate a code in Python function to validate an email address.

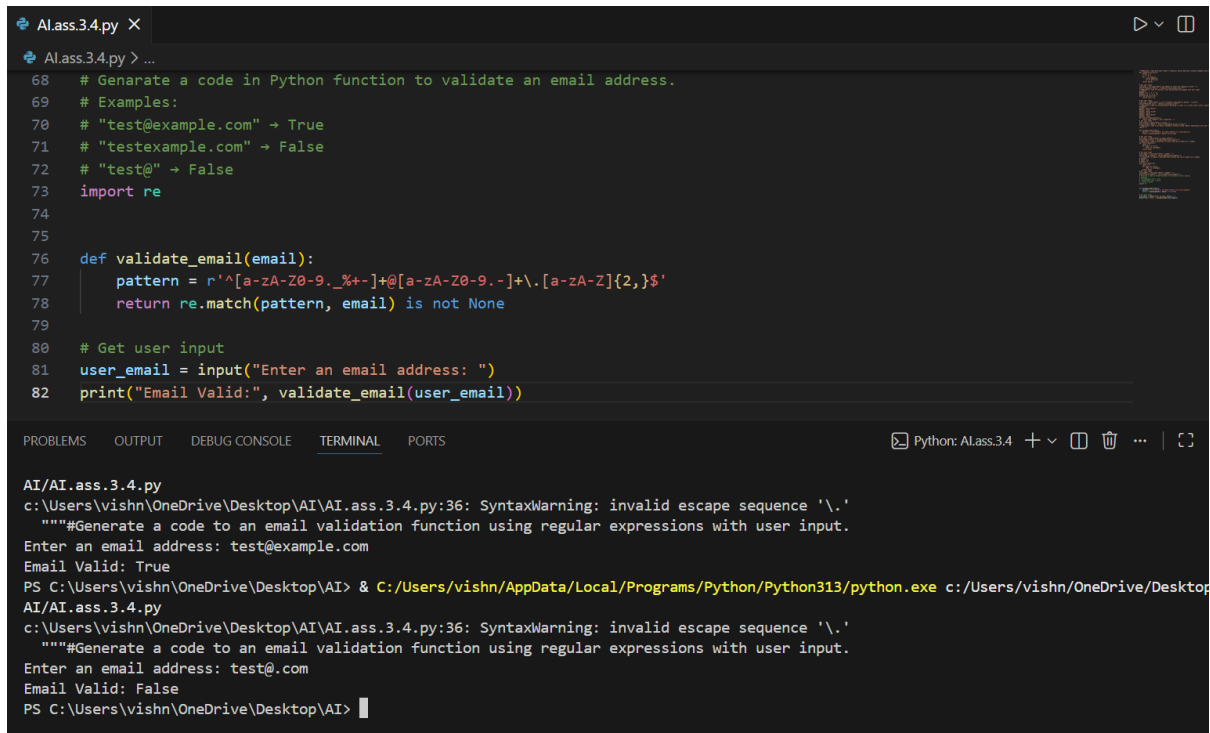
Examples:

"test@example.com" → True

"testexample.com" → False

"test@" → False

Code & Output:



```
AI.ass.3.4.py X
AI.ass.3.4.py > ...
68 # Generate a code in Python function to validate an email address.
69 # Examples:
70 # "test@example.com" → True
71 # "testexample.com" → False
72 # "test@" → False
73 import re
74
75
76 def validate_email(email):
77     pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
78     return re.match(pattern, email) is not None
79
80 # Get user input
81 user_email = input("Enter an email address: ")
82 print("Email Valid:", validate_email(user_email))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python: AI.ass.3.4 + - [] ...

AI/AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:36: SyntaxWarning: invalid escape sequence '\.'
"""#Generate a code to an email validation function using regular expressions with user input.
Enter an email address: test@example.com
Email Valid: True
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop
AI/AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:36: SyntaxWarning: invalid escape sequence '\.'
"""#Generate a code to an email validation function using regular expressions with user input.
Enter an email address: test@.com
Email Valid: False
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

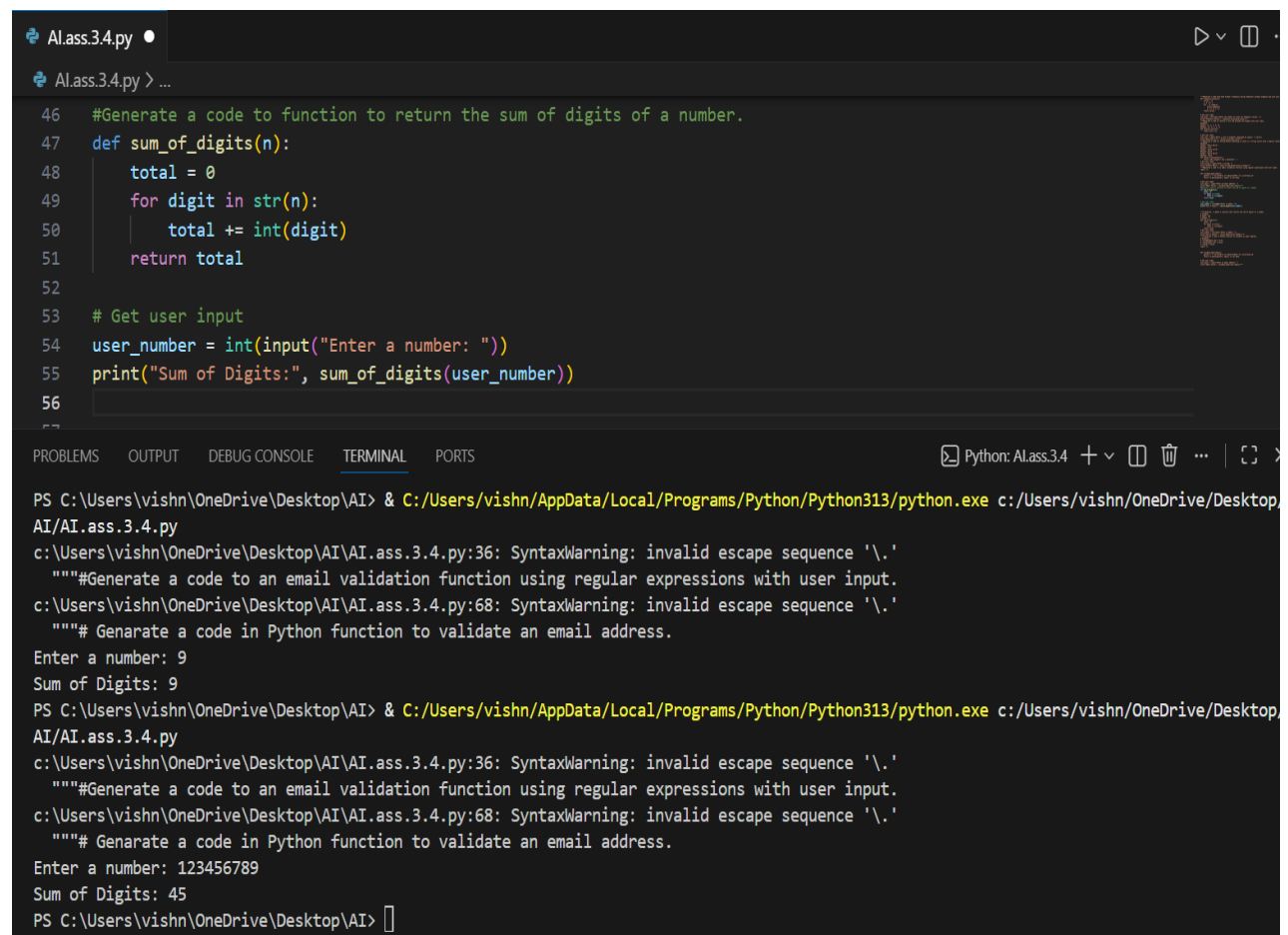
Observation: (Zero-shot vs Few-shot Prompt):

In the zero-shot prompt, GitHub Copilot generated a basic email validation function with simple checks, which may allow invalid email formats. When few-shot examples were provided, Copilot produced more detailed and structured validation logic, including proper checks for @ and domain parts. The few-shot approach reduced ambiguity and improved accuracy. This shows that providing examples makes the generated code more reliable and closer to real-world validation needs.

Task 5: Prompt Tuning – Summing Digits of a Number.

Prompt: #Generate a code to function to return the sum of digits of a number.

Code & Output:



The screenshot shows a Python IDE with a file named `AI.ass.3.4.py`. The code defines a function `sum_of_digits(n)` that calculates the sum of digits of a number `n`. It then prompts the user for input and prints the result. The terminal output shows two runs: one with input `9` resulting in `Sum of Digits: 9`, and another with input `123456789` resulting in `Sum of Digits: 45`. There are also syntax warnings in the terminal output.

```
46 #Generate a code to function to return the sum of digits of a number.
47 def sum_of_digits(n):
48     total = 0
49     for digit in str(n):
50         total += int(digit)
51     return total
52
53 # Get user input
54 user_number = int(input("Enter a number: "))
55 print("Sum of Digits:", sum_of_digits(user_number))
56
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python: AI.ass.3.4

```
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:36: SyntaxWarning: invalid escape sequence '\.'
    """#Generate a code to an email validation function using regular expressions with user input.
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:68: SyntaxWarning: invalid escape sequence '\.'
    """# Generate a code in Python function to validate an email address.
Enter a number: 9
Sum of Digits: 9
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:36: SyntaxWarning: invalid escape sequence '\.'
    """#Generate a code to an email validation function using regular expressions with user input.
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:68: SyntaxWarning: invalid escape sequence '\.'
    """# Generate a code in Python function to validate an email address.
Enter a number: 123456789
Sum of Digits: 45
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

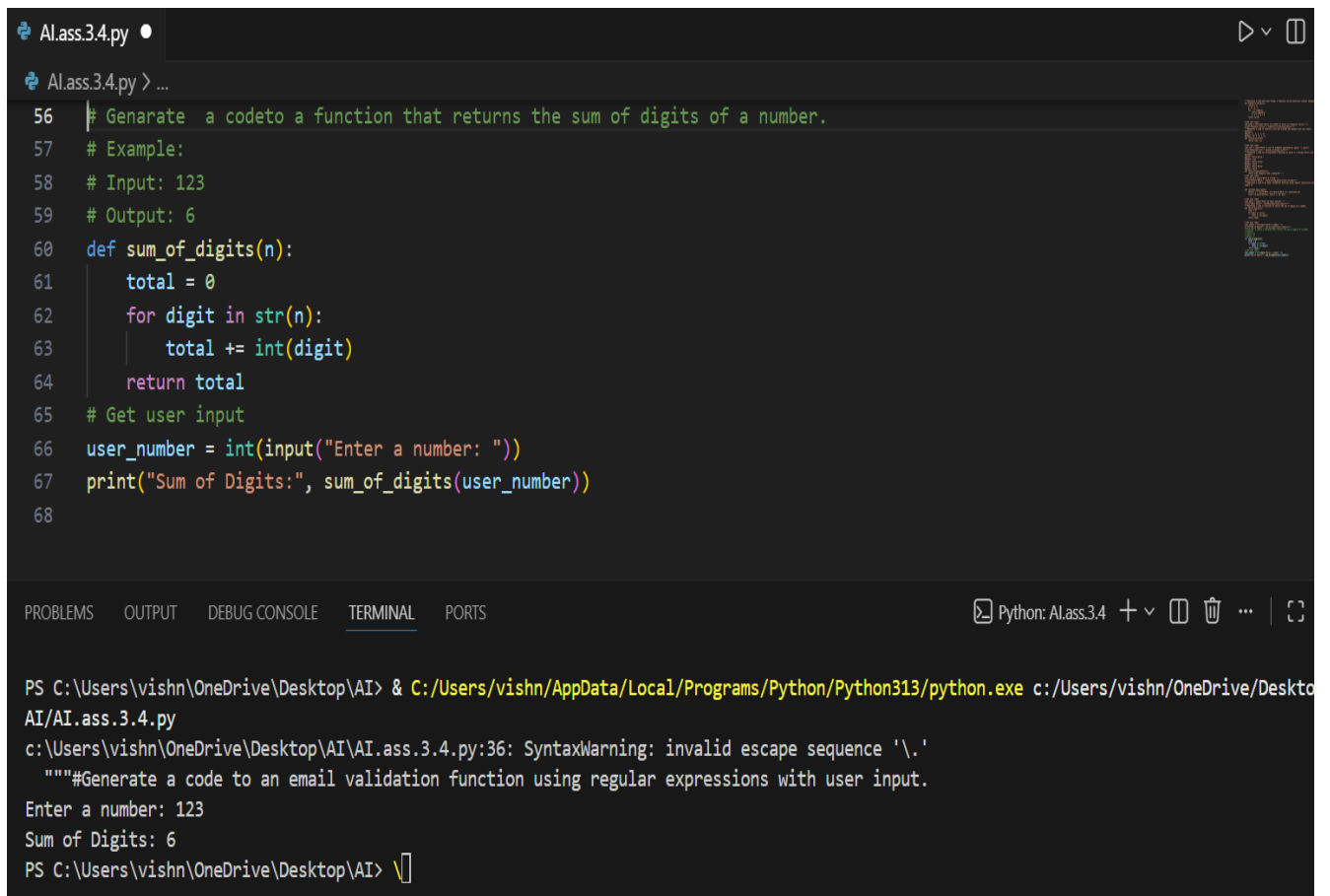
Prompt: # Genarate a codeto a function that returns the sum of digits of a number.

Example:

Input: 123

Output: 6

Code & Output:



The screenshot shows a VS Code editor window with a file named `AI.ass.3.4.py`. The code is a Python script that defines a function `sum_of_digits(n)` to calculate the sum of digits of a number. It includes a docstring with an example: `# Generate a code to a function that returns the sum of digits of a number. # Example: # Input: 123 # Output: 6`. The function uses a loop to iterate over the digits of the input number. The script also prompts the user for input and prints the result.

```
56 """Generate a code to a function that returns the sum of digits of a number.
57 # Example:
58 # Input: 123
59 # Output: 6
60 def sum_of_digits(n):
61     total = 0
62     for digit in str(n):
63         total += int(digit)
64     return total
65 # Get user input
66 user_number = int(input("Enter a number: "))
67 print("Sum of Digits:", sum_of_digits(user_number))
68
```

The terminal output shows the command to run the script, a syntax warning, the docstring, the user input `123`, and the output `Sum of Digits: 6`.

```
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python313/python.exe c:/Users/vishn/OneDrive/Desktop/AI/AI.ass.3.4.py
c:\Users\vishn\OneDrive\Desktop\AI\AI.ass.3.4.py:36: SyntaxWarning: invalid escape sequence '\.'
    """#Generate a code to an email validation function using regular expressions with user input.
Enter a number: 123
Sum of Digits: 6
PS C:\Users\vishn\OneDrive\Desktop\AI> \
```

Observation: (Prompt Tuning):

The generic prompt produced a correct but longer and more basic implementation of the `sum_of_digits()` function. When an input–output example was included, Copilot generated a cleaner and more optimized solution using concise Python syntax. The example clarified the expected behavior and encouraged better coding practices. This shows that prompt tuning with examples improves code quality and efficiency.